

0.54. Trois lettres, trois carrés

Le 0.53, plus une ligne : un C, avec les réglages rangés sous le nom « `ronde` ».

1. Ce qu'il faut comprendre

C'est le 0.53, plus un carré : le C, dont les réglages sont rangés en haut sous le nom `ronde`, comme au 0.48.

Ce qui est nouveau ici : rien qu'on ne connaisse, mais **ensemble** : deux carrés écrits en entier, et un troisième par son nom. `carre(ronde);` est remplacé par `carre(15, 8, ALPHABET[2], 2, 1, 250, 2);`.

Les trois centres sont assez loin l'un de l'autre pour que les carrés ne se touchent pas : (4, 4), (9, 9), (15, 8). À la fin, on voit les trois lettres.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Carre>` → [0.47.3. Carre — les réglages d'un carré sous un nom](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <carre>` → [0.33.3. La fonction carre\(\) — tourner en carré](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1411 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
carre()         440 o ← main() – une fonction fournie par le langage
main()          100 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
CopierTuiles    18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()  × 4    52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3    45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()  × 4    12 o sur place, dans carre(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
carre()  × 3    93 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Les réglages du C, rangés sous un nom (le 0.48) :  
//           x   y   tuile       taille  sens  vitesse  tours  
  
#include <Carre>      // les sept réglages d'un carré sous un seul nom  
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A  
#include <carre>      // une tuile qui tourne en carré
```

2. UNE VARIABLE — RONDE

```
Carre ronde = { 15,  8,  ALPHABET[2], 2,      1,   250,    2 };  
// autour de (15, 8), la lettre C, 5 × 5, sens des aiguilles, 250 ms par pas, 2 tours
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

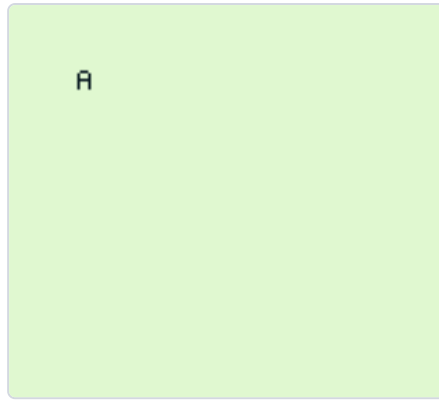
```
int main() {  
    // 1. Le A : autour de (4, 4) ; taille 1 (3 × 3) ; sens 1 ; 250 ms ; 1 tour.  
    carre(4, 4, ALPHABET[0], 1, 1, 250, 1);  
  
    // 2. Le B : autour de (9, 9) ; taille 3 (7 × 7) ; sens -1 ; 100 ms ; 1 tour.  
    carre(9, 9, ALPHABET[1], 3, -1, 100, 1);  
  
    // 3. La ligne ajoutée : le C, avec les réglages de « ronde ».  
    //    Le compilateur la remplace par carre(15, 8, ALPHABET[2], 2, 1, 250, 2).  
    carre(ronde);  
  
    while (true) {  
        image();           // A en (4, 4), B en (9, 9), C en (15, 8)  
    }  
}
```

3. Ce qui se passe, étape par étape

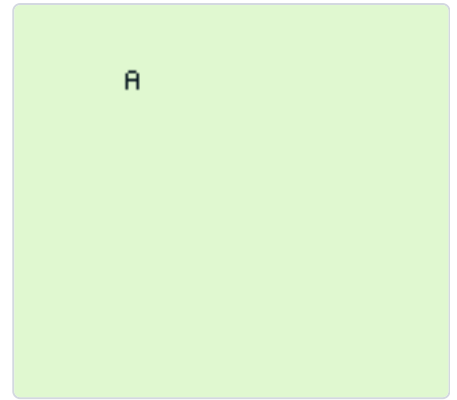
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



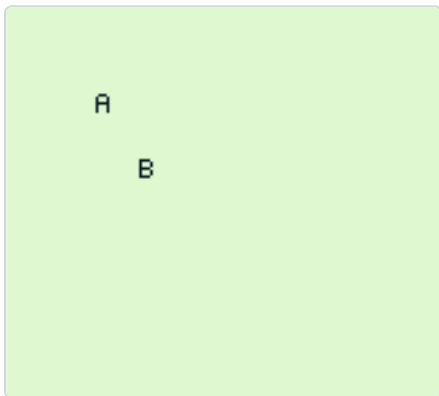
la toute première image — la console vient de démarrer — image 1



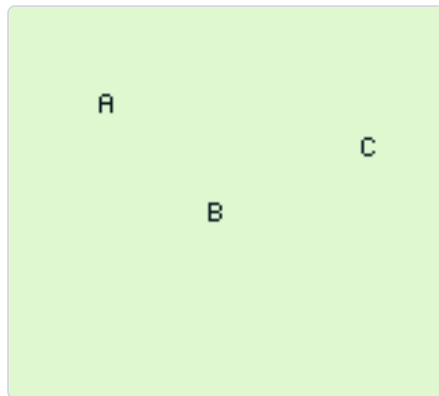
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A fait un petit carré, le B un grand carré rapide, puis le C deux tours autour de (15, 8). Chacun reste à son centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A et le B à leur centre : (4, 4), (9, 9)
- ✓ le C, réglé par « ronde » : deux tours 5 × 5 (2 passages au coin (17, 6))
- ✓ le C revient à son centre (15, 8) (15,8)

Ensuite : **0.54.1. Trois lettres, trois carrés — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++